

ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Пирамида тестирования

Модуль 3 · Лекция 5 · Уровни, пропорции, антипаттерны

Вера Терентьева · Lektion

4 курс

1 октября · Четверг 13:30–15:00

План лекции

01 Зачем нужна модель распределения тестов

03 Три уровня пирамиды: Unit, API/Integration, UI/E2E

05 Пропорции и экономика: 70/20/10, скорость, стоимость поддержки

07 Альтернатива: Testing Trophy

02 Проблема регрессионного тестирования

04 Тест-дизайн по уровням: black-box, white-box, experience

06 Антипаттерны: ice-cream cone и flaky-тесты

08 Как это устроено в Google и Lekton

Проблема: регрессионное тестирование

Регрессионное тестирование — проверка, что изменения не сломали существующую функциональность. Каждый коммит (новая фича, исправление бага, рефакторинг, обновление зависимости) несёт риск регрессии.

Скорость разработки выросла радикально. В современном CI/CD код мержится десятки раз в день. Если после каждого изменения гонять ручной регресс, разработка просто встанет.

Выход — автоматизация. Но автоматизировать всё тоже нельзя: тесты различаются по стоимости и скорости. Именно здесь на сцену выходит пирамида.

Пример СМП: масштаб ручного регресса

СМП учебный проект курса · система массового процессинга

11

микросервисов в СМП

+2

PostgreSQL и RabbitMQ

2–3 дня

один полный ручной регресс

Обработка одной транзакции задействует почти весь стек. При ежедневных изменениях полный ручной регресс физически невозможен.

Terminal
Simulator

Gateway

Switch

Authorization

Card
Management

Transaction
Logger

Синхронный путь + асинхронные цепочки через RabbitMQ (Transaction Logger, Notification Service)

Пирамида тестирования: происхождение и суть

Test Pyramid популяризовал **Майк Кон** в книге «Succeeding with Agile» (2009). Идея уровней не нова: ISTQB выделяет Component → Integration → System → Acceptance. Кон перевёл академическую классификацию в **экономическую метафору**.

E2E · ~10%

сквозные сценарии

Чем выше уровень, тем **меньше тестов**: внизу много дешёвых и быстрых, наверху мало дорогих и медленных.

API / Integration · ~20%

контракты между сервисами

Две независимые оси: масштаб (пирамида: unit → API → E2E) и источник информации (ISTQB: specification / structure / experience).

Unit · ~70%

логика в изоляции

Unit ≠ автоматически white-box. Категорию задаёт источник информации, а не уровень.

МОДУЛЬ 3 · ЛЕКЦИЯ 5

Три уровня пирамиды

Уровень 1: Unit-тесты

Проверяют **одну единицу кода в изоляции**: метод, функцию, класс. Внешние зависимости заменяются тестовыми двойниками.

Миллисекунды на тест · нет сети и БД · точная локализация дефекта прямо в IDE.

Тестовые двойники: stub — «что вернуть», mock — «был ли вызван».

Порядок: сначала **позитив** (валидное значение), затем границы и невалидные классы.

Два способа проектирования: по спецификации метода (чёрный ящик) или по структуре кода с требованием покрытия всех ветвей (белый ящик). Категорию задаёт источник информации, а не сам уровень.

Пример СМП: unit-тест валидатора Gateway

СМП Gateway · `TransactionRequestValidator.validate()`

Поле	Валидное	Невалидные классы
<code>mti</code>	<code>"0100"</code>	любое другое значение
<code>pan</code>	ровно 16 цифр	пусто, 15 цифр, 17 цифр
<code>amount</code>	<code>> 0</code>	0 и отрицательные
<code>currencyCode</code>	3 символа	иная длина
<code>mcc</code>	4 цифры	иная длина, не цифры
<code>stan, terminalId, merchantId</code>	обязательные	отсутствуют

Порядок: сначала позитив, затем границы и невалидные классы · **Техники модуля 2:** КЭ и граничные значения (PAN 15/17).

Уровень 2: Service / API / Integration

Проверяют **взаимодействие компонентов** через реальные интерфейсы: HTTP API, БД, очереди сообщений. Без UI и без полной картины системы.

Золотая середина: быстрее E2E, медленнее unit (сотни миллисекунд). Проверяют **контракты** — форматы запросов и ответов, статус-коды.

Негативные сценарии двух видов: **ошибки валидации** (400) и **бизнес-отказы** (200 + DECLINED). Порядок: сначала позитив (200 / APPROVED).

Ключевой вопрос: правильно ли сервис А вызывает сервис В? В микросервисной архитектуре это критично — каждый сервис живёт отдельно и общается через контракты. Тест держится за контракт, а не за реализацию.

Пример СМП: контрактный тест Authorization

СМП Authorization · `POST /api/internal/authorize`

Контракт `POST /api/internal/authorize` : принимает `AuthorizationRequest` , возвращает `AuthorizationResponse` . Тест на REST-assured держится за контракт, а не за реализацию — если поле переименуют или поменяют формат, тест упадёт именно там, где сломан договор между сервисами.

Сценарий	Вход	Ожидаемый ответ
Позитив	валидный <code>AuthorizationRequest</code>	200 · <code>mti=0110</code> · статус APPROVED или DECLINED
Негатив · валидация	PAN из 15 цифр	400 — сервис объявляет <code>@Valid</code> на теле запроса
Негатив · бизнес-отказ	запрос корректен, карта заблокирована/без средств	200 · статус DECLINED с кодом причины

Оба статуса допустимы в позитиве — важен факт корректного `AuthorizationResponse` .

Статус-коды как измерение: на каждый код (200/400/503) — свой тест.

Уровень 3: UI / E2E-тесты

Проверяют **систему целиком** через интерфейс или полный путь. Самые медленные (секунды и минуты), самые нестабильные и дорогие в поддержке.

flaky-тесты: падают случайно из-за сети, БД, таймингов, параллельных прогонов. Команды могут тратить до **20% времени** на их починку.

Проверяют то, что **не ловится ниже:** сквозные бизнес-сценарии. Таких тестов мало — только критичные.

Тест-дизайн от бизнес-сценариев: какие сквозные сценарии критичны для бизнеса и их нельзя проверить на нижних уровнях? Их немного — поэтому и E2E-тестов немного.

Пример СМП: E2E-тест асинхронного пути

СМП e2e-tests · RabbitMQAsyncE2eTest

Реальный тест `RabbitMQAsyncE2eTest` проверяет асинхронную доставку транзакции через RabbitMQ. Путь: Terminal Simulator → Gateway → Switch → Authorization → RabbitMQ → Transaction Logger → Dashboard. Один тест проверяет связность всего контура — то, что нельзя поймать ни на unit, ни на API-уровне.

Сценарий	Что проверяем
TC-22 · eventual consistency	транзакция отправляется через Gateway (<code>POST /api/transactions</code> → 200), затем <code>Awaitility</code> опрашивает поиск по STAN, пока запись не появится в Logger (до 15 сек)
TC-23 · сбой RabbitMQ	при недоступности брокера для APPROVED-транзакции срабатывает откат резервирования (rollback)

Eventual consistency: запись появляется с задержкой — ждём условие, а не фиксированное время.

Почему дорого: зависит от сети, БД, таймингов — flaky. Таких тестов немного.

Тест-дизайн по уровням: две независимые оси

Масштаб (пирамида: unit → API → E2E) и **источник информации** (ISTQB: specification / structure / experience) — это **две разные оси**. Их нельзя смешивать: пирамида отвечает «какой масштаб проверки», а категория — «откуда взять проверку».

Уровень	Категория (источник)	Основные техники
Unit	structure-based, частично specification-based	покрытие ветвей, условий, путей; КЭ и ГЗ на параметры метода
Service / API	specification-based (black-box)	КЭ, ГЗ, таблицы решений, проверка статус-кодов, позитив/негатив
E2E	black-box + experience-based	сценарии бизнес-процессов, state-transition, исследовательское тестирование

1. Выбрать уровень — самый низкий

2. Определить источник

3. Применить технику из модуля 2

Одна операция — разные уровни: создание карты

СМП Card Management · `POST /api/cards` → создание карты

Идейно одна операция — «создать карту» — распадается на несколько проверок, и каждая живёт на своём уровне пирамиды. Уровень выбираем не по названию операции, а по тому, какой риск ловим и насколько дешёва проверка.

E2E · событие до Notification

outbox → RabbitMQ → consumer

API · контракт `POST /api/cards`

201 / 400 / 409, тело Card

Unit · логика без сети и БД

Одна операция — три уровня. Создание карты не привязано к единственному уровню: у него есть чистая логика, есть контракт и есть сквозная доставка события.

Каждый уровень ловит свой риск. Луна ловит ошибку контрольной цифры, контракт — рассинхрон форматов, E2E — потерю события в очереди.

МОДУЛЬ 3 · ЛЕКЦИЯ 5

Экономика пирамиды

Пропорции: правило 70/20/10

~70%

Unit-тесты — логика в изоляции

~20%

Service / Integration —
контракты

~10%

E2E — сквозные сценарии

Это ориентир, а не догма. Enterprise с плотными интеграциями (как Lekton) может поднять долю service-тестов до 30–40%: именно там живут основные интеграционные риски.

Риск стартапа с простым бэкендом и сложным UI — скатиться в ice-cream cone: слишком много E2E-тестов. Разберём этот антипаттерн чуть позже.

Важнее процентов принцип: каждый сценарий тестируйте на **самом низком уровне**, который способен его адекватно проверить. Нужна только логика — unit. Нужен контракт между сервисами — API. Нужна вся цепочка — E2E.

Экономика: скорость обратной связи



Unit

10–100 мс · результат за секунды



Service / API

100–500 мс · результат за минуты



E2E

1–30 сек · результат за десятки минут

Правило 1 : 10 : 100. Дефект, пойманный на unit-уровне, стоит условную единицу; на интеграционном — 10; на E2E — 100; в продакшне — 1000. Скорость обратной связи напрямую определяет стоимость дефекта.

Психология потока. Если тесты идут 20 минут, разработчик переключится на другую задачу и потеряет контекст. Обратная связь за секунды поддерживает концентрацию.

Экономика: стоимость поддержки

Что ломает тест

Unit — только изменение логики метода

API — изменение контракта (формат запросов и ответов)

E2E — любое изменение: UI, контракт, данные, тайминги

Что это значит

Чем выше уровень, тем выше стоимость сопровождения

Парадокс пестицида: набор нужно регулярно обновлять

E2E — только там, где сценарий нельзя проверить ниже

Каждый лишний E2E — долг по сопровождению на годы вперёд

МОДУЛЬ 3 · ЛЕКЦИЯ 5

Антипаттерны и альтернативы

Антипаттерн: ice-cream cone

Перевернутая пирамида: много E2E/UI-тестов, мало unit. Причина — команда начинает автоматизацию «сверху»: ручные тестировщики переходят в автоматизацию через UI.

Симптом: «200 E2E-тестов, идут 45 минут, 15 из них flaky». Команда перестаёт доверять CI: «красный билд, ну это опять flaky».

Лечение: постепенно переносим проверки вниз по пирамиде: «кнопка вызывает POST» и «отрицательная сумма отклоняется» — в unit, а в E2E оставляем только сквозной сценарий.

Быстрый старт с долгим торможением: UI-тесты наглядны — их легко показать руководству. Но каждый UI-тест — это часы сопровождения в будущем и часы прогона в CI.

Много UI и E2E

Немного API

Почти нет
unit

Рожок мороженого: широкая вершина, узкое основание

Альтернатива: Testing Trophy

Kent C. Dodds, 2018: «Write tests. Not too many. Mostly integration». Фокус — на интеграционных тестах как основном уровне.

Аргумент против перекоса в unit: unit-тесты проверяют реализацию, а не поведение. Хороший рефакторинг может сломать десятки unit-тестов — это **brittle-тесты**.

Статика (линтеры, компилятор, система типов) — дешёвая замена части тривиальных unit-проверок.

Терминологическая ловушка: «integration» в Trophy (компоненты вместе, без реальных БД и сети) ≠ «интеграционное тестирование» в модуле 6 (реальные зависимости в контейнерах).



Кубок: узкое основание из статика, широкий средний ярус

Test Pyramid vs Testing Trophy

Критерий	Test Pyramid	Testing Trophy
Основание	Много unit-тестов	Статические проверки (линтеры, типы)
Основной ярус	Unit (~70%)	Integration (основной уровень)
Пропорции	70% unit / 20% integration / 10% E2E	Статика → integration → немного unit → мало E2E
Фокус	Дешёвая изоляция	Проверка поведения
Общее	E2E всегда мало, обратная связь быстрая	E2E всегда мало, обратная связь быстрая
Когда выбирать	Монолит с чистой бизнес-логикой	Микросервисы и фронтенд
«Integration» означает	Тесты компонентов через реальные интерфейсы	Компоненты вместе, без реальных БД и сети
В СМП	Unit для Authorization и валидации Gateway	API-тесты контрактов, как в Trophy

Общее у обеих моделей: E2E всегда мало, обратная связь быстрая. Спор — о доле unit и доверии к статике.

Как это работает в реальных компаниях

Компания	Уровни	Пропорции	Когда запускается
Google	Small / Medium / Large (таксономия)	80% Small · 15% Medium · 5% Large	Small — pre-commit, Medium — CI, Large — nightly
Lekton	Unit → Модульные (Java) → Интеграционные → E2E + Системное → Клиентские конфигурации	по скорости прогона	быстрые чаще, медленные — реже
Общее правило	Группировка тестов по времени выполнения	unit → CI → nightly	Pre-commit: unit · CI: unit + integration · Nightly: E2E + нагрузочные

Интерактив: распределение сценариев по уровням

1

PAN из 15 цифр отклоняется

Валидация входных данных

2

Полный путь транзакции

Сквозной сценарий от входа до лога

3

Достаточность баланса

Бизнес-логика Authorization

4

Неизвестный BIN и PAN

Отказ по BIN и 404 по карте

Вопрос: на каком уровне тестировать каждый сценарий и почему? Уровень, категория, порядок проверок.

Интерактив: разложи создание карты по уровням

1

Контрольная цифра PAN

Алгоритм Луна

2

expiryDate = сегодня + 3 года

Вычисление срока действия

3

Контракт POST /api/cards

201 / 400 / 409 и тело Card

4

Событие доходит до Notification

outbox → RabbitMQ → consumer

Вопрос: на каком уровне тестировать каждую проверку и почему? Уровень, категория по источнику информации, порядок проверок.

Выводы и анонс



Пирамида — экономическая модель

Уровни: unit, API, E2E.



Главный принцип

Сценарий — на самом низком уровне. 70/20/10 — ориентир.



Враги пирамиды

Ice-cream cone и flaky-тесты.



Метод выбора

Уровень → источник информации → техника из модуля 2.

Порядок: сначала позитив, затем негатив.

Лекция 6: основы CI/CD — запуск тестов в пайплайне.

Источники

Cohn, M. (2009). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley. — Глава 15 (Testing), Test Pyramid.

Dodds, K. C. (2018). «Write tests. Not too many. Mostly integration». kentcdodds.com/blog/write-tests — Testing Trophy.

Winters, T., Manshreck, T., & Wright, H. (2020). *Software Engineering at Google*. O'Reilly. — Глава 11 (Testing Overview), таксономия Small/Medium/Large.

Spillner, A., Linz, T., & Schaefer, H. (2014). *Software Testing Foundations* (4th ed.). Rocky Nook. — Главы 2.4, 3, 6.3.1.

Myers, G., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). Wiley. — Глава 5 (Test-Case Design).

Bach, J., & Bolton, M. (2025). *Taking Testing Seriously: The Rapid Software Testing Methodology*. Wiley. — Глава 6 (Automation).

ISTQB — Certified Tester Foundation Level (CTFL) v4.0, Syllabus и Glossary. — Категории техник: specification-based, structure-based, experience-based; разграничение уровней; практический гибрид «серый ящик».

The background is a dark blue gradient. It features several stylized, glowing green classical buildings with columns, resembling temples or banks. These buildings are interconnected by a network of glowing green lines that resemble circuitry or data flow. There are also several glowing green rectangular shapes, possibly representing tablets or screens, scattered throughout the scene.

Вопросы?